Clasificación de dígitos manuscritos mediante técnicas de aprendizaje automático y redes neuronales

Sara Santos García

24 de julio de 2026

Resumen

El presente trabajo aborda el problema del reconociento de caractéres manuscritos. Se comparan varios modelos clásicos de aprendizaje automático (k-Nearest Neighbors y Máquinas de Soporte Vectorial), técnicas de reducción de dimensionalidad (Análisis de Componentes Principales), una red neuronal con una capa oculta (perceptrón multicapa), y una red neuronal convolucional (CNN). El objetivo es analizar los diferentes resultados, optimizar la red neuronal y determinar la necesidad de usar una red convolucional a medida que el problema asciende en nivel de dificultad.

1. Descripción del problema y conjuntos de datos

El problema elegido es el reconocimiento de caracteres manuscritos, por ser un problema clásico en el campo de la computación y ser el ejemplo presentado en el manual. Mi idea es inicialmente usar los parámetros tal y como se nos describen en dicho manual y seguidamente realizar las modificaciones pertinentes y así poder analizar los cambios en la precisión de los distintos modelos.

En el caso de los modelos de aprendizaje automático clásicos y de redes neuronales profundas se ha usado el conjunto de datos `digits`, de la biblioteca Scikit-Learn. Este *dataset* cuenta con 1.797 imágenes de 8×8 píxeles en escala de grises.

El primer paso en todos los casos ha sido reordenar estas matrices de 64 elementos como vectores unidimensionales para El vector objetivo Y contiene etiquetas con los dígitos del 0 al 9, que se corresponden con cada uno de los dígitos a reconocer, lo que define esta tarea como un problema de clasificación multiclase. Por tanto, en el caso de redes neuronales, tendremos que tener como mínimo 10 neuronas de salida y más de una capa, pues una neurona resuelve un problema de clasificación binario.

Los datos se dividen aleatoriamente en un conjunto de entrenamiento y un conjunto de test, de forma que el algoritmo o la red no memoriza, se enfrenta a unos datos que no conocía a la hora de evaluar.

A mayores, se ha decididio incluir una red neuronal convolucional para resolver el mismo problema pero con el dataset `MNIST`, cuyas imágenes de 28×28 píxeles requieren a priori otras técnicas más avanzadas. Este conjunto de datos consta de 60.000 imágenes de entrenamiento y 10.000 imágenes de prueba, mucho más amplio que `digits`. Se ha decidido también resolver este problema con la red neuronal anteriormente mencionada para determinar si la convolucional es realmente necesaria.

2. Técnicas aplicadas y justificación de diseño

Para establecer una comparativa rigurosa entre modelos clásicos de aprendizaje automático y redes neuronales se han usado k-Nearest Neighbours, máquinas de soporte vectorial (SVM) y

análisis de componentes principales (PCA), que se van a comparar con una red neuronal de dos capas.

2.1. k-Nearest Neighbors (k-NN)

Como modelo inicial se ha usado el algoritmo no parámetrico de los k vecinos más cercanos, que asume que las muestras de una misma clase se sitúan en regiones cercanas. Se ha fijado el hiperparámetro de $k = 3$, un valor bajo, para evitar cruzar las fronteras de decisión, y se ha usado la métrica euclídea (la distancia usual).

2.2. Máquinas de Soporte Vectorial (SVM)

Las SVM son otro tipo de clasificadores, en este caso paramétricos, que buscan un hiperplano de separación de máximo margen.

Se han evaluado dos variantes para estudiar el impacto de la no linealidad del problema, un modelo SVC lineal frente a un SVC polinómico. En concreto, se justifica el uso de un kernel polinómico de grado 2 para transformar el espacio de entrada, presumiendo que las combinaciones no lineales separan mejor los dígitos que se parecen (como el 3 y el 8).

2.3. Análisis de Componentes Principales (PCA)

Como el espacio de características original consta de 64 dimensiones, se aplica PCA como algoritmo de aprendizaje no supervisado para la reducción de dimensionalidad.

El pre-procesamiento incluye el centrado de los datos a media cero (necesario para este algoritmo). Se seleccionó retener $k = 20$ componentes, optando por un equilibrio entre reducir el coste computacional y preservar la información de las imóagenes.

Posteriormente, se evaluó una SVM polinómica sobre este espacio reducido.

2.4. Red neuronal con 1 capa oculta o Perceptrón Multicapa (MLP)

Aquí se aborda el caso de las redes neuronales artificiales profundas mediante Keras. Se ha usado como arquitectura inicial la presentada en el manual del curso. Esta consta de una capa de entrada (64 características), una capa oculta (100 neuronas con activación sigmoide) y una capa de salida (10 neuronas). Se aplicó *one-hot encoding* al vector objetivo y se utilizó la función de activación *softmax* en la salida para interpretar las predicciones como un vector de probabilidades. El entrenamiento se guió minimizando el error cuadrático medio (MSE) mediante *Mini-Batch Stochastic Gradient Descent* (SGD) durante 3000 épocas con un subconjunto de 32 muestras.

Se ha decidido realizar diferentes cambios para optimizar el funcionamiento de la red neuronal. En primer lugar, se ha modificado la función de optimización de la función de error, con RMSprop (*Root Mean Square Propagation*). Además, se han reducido las *epochs* pues la rápida convergencia de RMSprop lo permitía, y se han probado diferentes funciones de activación de la primera capa de neuronas, entre ellas *sigmoide, tanh, softmax* y *ReLu*. Detalles en la sección de resultados y análisis crítico.

Para realizar una comparación directa con una red convolucional, se ha decidido adaptar el dataset `MNIST` del siguiente apartado a esta red monocapa también.

2.5. Red convolucional

Para el ejemplo de red convolucional se ha usado el dataset `MNIST`, de imágenes de dígitos de 28×28 pues aquí es donde brillan este tipo de redes, al añadir complejidad y tamaño a las imágenes. Se ha usado un algoritmo de convolución (con *ReLu* de activación), seguido de un pooling y de un flatten, tras lo cual actúa una red neuronal con 1 capa oculta.

Como hiperparámetros se han seleccionado la función de error *categorical crossentropy* optimizada por SGD (ya vimos que no era óptima), funciones de activación de la red *ReLu* y *softmax*, y un *batch size* de 100, lo que da 600 iteraciones por época.

3. Resultados y comparación de los modelos

El resultado del algoritmo k-NN es de `0.97555`.
El resultado del algoritmo SVM lineal es de `0.94888`.
El resultado del algoritmo SVM polinómico es de `0.99555`.
El resultado del algoritmo PCA + SVM es de `0.98222`.
El resultado de la red neuronal con los hiperparámetros mencionados, tras 8 minutos, es

```
Epoch 1/3000 34/34 ---------------- 1s 3ms/step - accuracy: 0.1271 - loss: 0.0955
Epoch 2/3000 34/34 ---------------- 0s 3ms/step - accuracy: 0.1308 - loss: 0.0950
Epoch 3/3000 34/34 ---------------- 0s 2ms/step - accuracy: 0.1317 - loss: 0.0945
Epoch 4/3000 34/34 ---------------- 0s 3ms/step - accuracy: 0.1317 - loss: 0.0941
Epoch 5/3000 34/34 ---------------- 0s 2ms/step - accuracy: 0.1336 - loss: 0.0937
Epoch 6/3000 34/34 ---------------- 0s 3ms/step - accuracy: 0.1354 - loss: 0.0933
Epoch 7/3000 34/34 ---------------- 0s 3ms/step - accuracy: 0.1391 - loss: 0.0929
Epoch 8/3000 34/34 ---------------- 0s 2ms/step - accuracy: 0.1391 - loss: 0.0925
Epoch 9/3000 34/34 ---------------- 0s 3ms/step - accuracy: 0.1410 - loss: 0.0922
Epoch 10/3000 34/34 ---------------- 0s 3ms/step - accuracy: 0.1419 - loss: 0.0918
Epoch 11/3000 34/34 ---------------- 0s 2ms/step - accuracy: 0.1456 - loss: 0.0915
Epoch 12/3000 34/34 ---------------- 0s 2ms/step - accuracy: 0.1475 - loss: 0.0912
...
Epoch 3000/3000 34/34 ---------------- 0s 3ms/step - accuracy: 0.9972 - loss: 0.0025
        8/8 ---------------- 0s 7ms/step - accuracy: 0.9624 - loss: 0.0072
        [0.007162821479141712, 0.9624478220939636]
```

Los podemos observar resumidamente en la siguiente tabla

Modelo evaluado	**Tasa de acierto (Test)**
k-Nearest Neighbors ($k = 3$)	$\approx 97{,}6\,\%$
SVM (Lineal)	$\approx 94{,}9\,\%$
SVM (Polinómico, $d = 2$)	$\approx 99{,}6\,\%$
PCA ($k = 20$) + SVM Polinómico	$\approx 98{,}2\,\%$
Perceptrón Multicapa (Keras)	$\approx 98{,}3\,\%$
Red convolucional 28x28	$\approx 96{,}7\,\%$
Red neuronal 28x28	$\approx 89\,\%$

Cuadro 1: Comparativa de la eficacia de los modelos implementados.

3.1. Optimización de la red neuronal multicapa

Observamos que la convergencia es lenta, y aunque en el entrenamiento se alcanza una tasa de acierto de casi el 100 %, a la hora de evaluar esa tasa baja al 96,2 %. Esto probablemente sea fruto del sobre-entrenamiento y el *vanishing gradient problem*, al que la función sigmoide no es inmune. En cambio, usando RMSprop como función de optimización, obtenemos, tras solo 100 *epochs*,

```
Epoch 1/100 34/34 ---------------- 1s 3ms/step - accuracy: 0.3868 - loss: 0.0771
Epoch 2/100 34/34 ---------------- 0s 4ms/step - accuracy: 0.7217 - loss: 0.0524
Epoch 3/100 34/34 ---------------- 0s 3ms/step - accuracy: 0.8451 - loss: 0.0366
Epoch 4/100 34/34 ---------------- 0s 2ms/step - accuracy: 0.9017 - loss: 0.0268
```

```
Epoch 5/100 34/34 ---------------- 0s 3ms/step - accuracy: 0.9286 - loss: 0.0205
Epoch 6/100 34/34 ---------------- 0s 3ms/step - accuracy: 0.9388 - loss: 0.0162
Epoch 7/100 34/34 ---------------- 0s 2ms/step - accuracy: 0.9536 - loss: 0.0130
Epoch 8/100 34/34 ---------------- 0s 3ms/step - accuracy: 0.9555 - loss: 0.0110
Epoch 9/100 34/34 ---------------- 0s 3ms/step - accuracy: 0.9685 - loss: 0.0093
Epoch 10/100 34/34 ---------------- 0s 2ms/step - accuracy: 0.9694 - loss: 0.0081
Epoch 11/100 34/34 ---------------- 0s 3ms/step - accuracy: 0.9740 - loss: 0.0071
Epoch 12/100 34/34 ---------------- 0s 3ms/step - accuracy: 0.9777 - loss: 0.0062
Epoch 13/100
...
Epoch 100/100 34/34 ---------------- 0s 2ms/step - accuracy: 1.0000 - loss: 9.2405e-05
       8/8 ---------------- 0s 6ms/step - accuracy: 0.9666 - loss: 0.0049
       [0.004873801954090595, 0.9666203260421753]
```

Se observa claramente una convergencia mucho más rápida en los primeros pasos, aunque hasta cierto punto se sigue observando un problema de sobre-entrenamiento pues la tasa de acierto baja respecto a la de acierto en entrenamiento (seguimos usando sigmoide como activación en la capa 1).

Finalmente, usando como función de activación ReLu en la 1ª capa, obtenemos

```
Epoch 1/100 34/34 ---------------- 1s 3ms/step - accuracy: 0.3312 - loss: 0.0992
Epoch 2/100 34/34 ---------------- 0s 3ms/step - accuracy: 0.5909 - loss: 0.0586
Epoch 3/100 34/34 ---------------- 0s 3ms/step - accuracy: 0.6948 - loss: 0.0431
Epoch 4/100 34/34 ---------------- 0s 3ms/step - accuracy: 0.7403 - loss: 0.0371
Epoch 5/100 34/34 ---------------- 0s 2ms/step - accuracy: 0.8302 - loss: 0.0248
Epoch 6/100 34/34 ---------------- 0s 2ms/step - accuracy: 0.8590 - loss: 0.0201
Epoch 7/100 34/34 ---------------- 0s 2ms/step - accuracy: 0.8701 - loss: 0.0183
Epoch 8/100 34/34 ---------------- 0s 2ms/step - accuracy: 0.8748 - loss: 0.0171
Epoch 9/100 34/34 ---------------- 0s 2ms/step - accuracy: 0.8813 - loss: 0.0162
Epoch 10/100 34/34 ---------------- 0s 2ms/step - accuracy: 0.9221 - loss: 0.0109
Epoch 11/100 34/34 ---------------- 0s 3ms/step - accuracy: 0.9545 - loss: 0.0069
Epoch 12/100 34/34 ---------------- 0s 2ms/step - accuracy: 0.9638 - loss: 0.0055
...
Epoch 100/100 34/34 ---------------- 0s 3ms/step - accuracy: 1.0000 - loss: 3.2713e-06
       8/8 ---------------- 0s 5ms/step - accuracy: 0.9833 - loss: 0.0026
       [0.00258587091229856, 0.9833101630210876]
```

con una convergencia similar al caso anterior pero con una tasa de acierto de casi un 2 % mayor, lo cual dependiendo de la situación puede ser fundamental. Este mejor resultado puede explicarse por la inmunidad de *ReLu* al problema de desvanecimiento de gradiente, aunque no debería ser de gran importancia en nuestro caso, pues solo tenemos una capa de neuronas intermedias.

Para hacer una última comparación, se ha modificado la función de activación de la primera capa y las *epoch* casi al límite de lo que permite la convergencia, con 10 iteraciones obtenemos los siguientes resultados:

Función evaluada	**Tasa de acierto (test)**
Sigmoide	$\approx 95{,}7\,\%$
Tanh	$\approx 97{,}1\,\%$
Softmax	$\approx 55{,}4\,\%$
çtesxtitReLu	$\approx 95{,}6\,\%$

Cuadro 2: Comparativa de la eficacia de las funciones de activación.

Se concluye pues que en esta tarea y con esta red neuronal la importancia de la función de optimización de la *loss function* es crucial. Esta nos permite optimizar los recursos para entrenar el modelo y reducir la carga computacional. Vemos también que la función de activación tiene su peso aunque no es tan crucial en el caso de una red neuronal de dos capas. Y por último, observamos que comparando con los resultados con 100 *epochs*, la red aprende muy rápido al principio y más lento al final.

3.2. Resultados de la red convolucional

Tras casi 4 minutos, la salida de la red convolucional es

```
Epoch 1/10 600/600 ---------------- 23s 35ms/step - accuracy: 0.8117 - loss: 0.7432
Epoch 2/10 600/600 ---------------- 19s 32ms/step - accuracy: 0.9153 - loss: 0.2930
Epoch 3/10 600/600 ---------------- 23s 35ms/step - accuracy: 0.9306 - loss: 0.2388
Epoch 4/10 600/600 ---------------- 20s 33ms/step - accuracy: 0.9412 - loss: 0.2021
Epoch 5/10 600/600 ---------------- 26s 44ms/step - accuracy: 0.9491 - loss: 0.1768
Epoch 6/10 600/600 ---------------- 42s 45ms/step - accuracy: 0.9539 - loss: 0.1569
Epoch 7/10 600/600 ---------------- 34s 32ms/step - accuracy: 0.9585 - loss: 0.1410
Epoch 8/10 600/600 ---------------- 20s 31ms/step - accuracy: 0.9626 - loss: 0.1282
Epoch 9/10 600/600 ---------------- 18s 29ms/step - accuracy: 0.9659 - loss: 0.1169
Epoch 10/10 600/600 ---------------- 22s 31ms/step - accuracy: 0.9681 - loss: 0.1079
        313/313 ---------------- 2s 5ms/step - accuracy: 0.9698 - loss: 0.0989
        [0.09888315200805664, 0.9697999954223633]
```

y se observa que el resultado del porcentaje de acierto en el test es incluso mayor que en el entrenamiento. Esto nos dice que claramente la red ha aprendido a resolver el problema, no se ha sobre-entrenado y no ha memorizado conexiones entre neuronas.

4. Análisis crítico

Gracias al Cuadro 1, observamos que para este problema de imágenes 8×8 no es necesario recurrir a las redes multicapa y que otros modelos de aprendizaje automático son también adecuados para su resolución. Sin embargo, probablemente se deba a la relativa sencillez del problema a resolver. En el momento en que el problema requiera una mayor complejidad, como imágenes de mayor calidad, detección de personas en vídeo o problemas similares, las redes neuronales optimizan la tarea e infieren los ejemplos de entrenamiento mejor que las técnicas de aprendizaje automático, sobre todo las de tipo convolucional.

Si intentamos resolver el problema con el dataset `MNIST` y la red neuronal con una capa oculta, la convergencia es significativamente más lenta y los recursos necesarios mucho mayores (89 % de acierto donde la red convolucional conseguía un 97 % con las mismas *epochs*). Por otro lado, aumentando el número de capas y las neuronas de estas se ha llegado a conseguir la misma tasa de acierto con la red neuronal. Quedaría pendiente contrastar el coste computacional de ambos entrenamientos.

Se concluye así que cada problema requiere de un algoritmo diferente, y que debemos optimizar nuestras herramientas para tampoco caer en un *overkill*, o como diríamos aquí, para no "matar moscas a cañonazos" (como usar la red convolucional para reconocer imágenes de 8×8). Pero tampoco debemos caer en un subajuste o *underfitting*, como sería el caso de usar la red neuronal monocapa para el dataset `MNIST`.